COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/10//2026	**DATE SUBMITTED:**	09/18/2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 — **Sorting Algorithms Pt2 / SORTING ALGORITHMS: SHELL, MERGE, AND QUICK SORT**

1. PROCEDURE OUTPUT

What procedures did you perform?

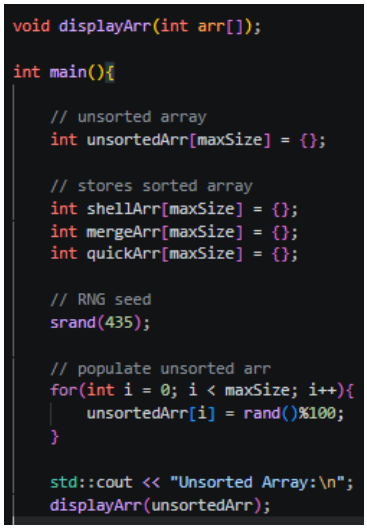
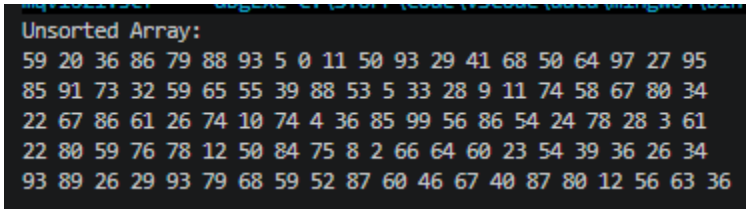
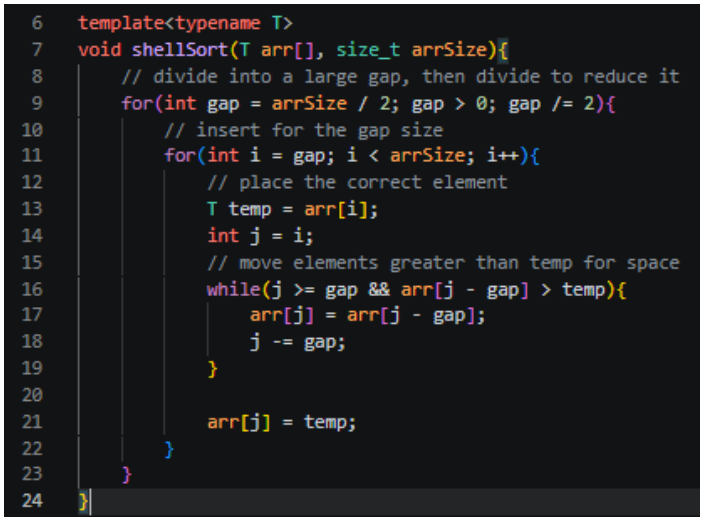
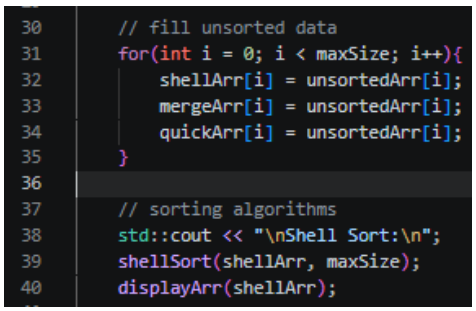
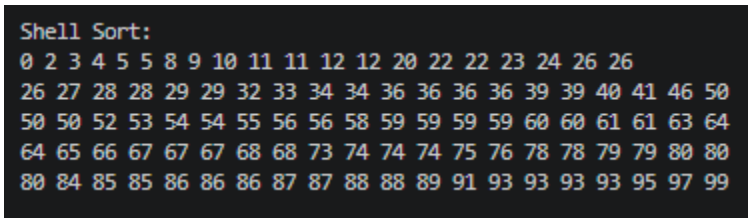
Code + Console Screenshot	  
Observations	The driver code uses a pseudorandom number generator with a predetermined seed to create an unsorted array of random values, so the sorting algorithms can use the data to perform their sorting procedures. The four declared arrays are used for storing the values of the generated number. The generated unsorted array in the output is the result of the driver code.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	  
Observations	The shell sort technique first starts with a large gap and reduces its size by dividing it for each iteration. The inner loop performs an insertion sort based on the gap size, and then it moves elements that are greater than temp to create space. After this, the temporary elements are placed in their correct position. The output shows the result of the shell sort technique in one function in the header file. The best case time complexity when the unsorted array is mostly sorted is O(n log n). The worst case scenario is O(n^2) due to the inner loops and dividing gap sequences.

```
Shell Sort:
0 2 3 4 5 5 8 9 10 11 11 12 12 20 22 22 23 24 26 26
26 27 28 28 29 29 32 33 34 34 36 36 36 36 39 39 40 41 46 50
50 50 52 53 54 54 55 56 56 58 59 59 59 59 60 60 61 61 63 64
64 65 66 67 67 67 68 68 73 74 74 74 75 76 78 78 79 79 80 80
80 84 85 85 86 86 86 87 87 88 88 89 91 93 93 93 93 95 97 99
```

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```cpp
template<typename T>
void merge(T arr[], int left, int middle, int right){
    int n1 = middle - left + 1;
    int n2 = right - middle;

    // temp arrays
    T L[n1];
    T R[n2];

    // copy array data to temp arrays
    for(int i = 0; i < n1; i++){
        L[i] = arr[left + i];
    }

    for(int j = 0; j < n2; j++){
        R[j] = arr[middle + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = left;

    // merge temp arrays into [left...right]
    while(i < n1 && j < n2){
        if(L[i] <= R[j]){
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
```

```cpp
    // copy L[] and R[] remaining elements if any remain
    while(i < n1){
        arr[k] = L[i];
        i++;
        k++;
    }

    while(j < n2){
        arr[k] = R[j];
        j++;
        k++;
    }
}
```

```
75  template<typename T>
76  void mergeSort(T arr[], int left, int right){
77      if(left < right){
78          int middle = (left + right) / 2;
79          // sort first half recursively
80          mergeSort(arr, left, middle);
81          // 2nd half
82          mergeSort(arr, middle + 1, right);
83          // merge
84          merge(arr, left, middle, right);
85      }
86  }
```

```
42      std::cout << "\nMerge Sort:\n";
43      mergeSort(mergeArr, 0, maxSize - 1);
44      displayArr(mergeArr);
```

```
Merge Sort:
0 2 3 4 5 5 8 9 10 11 11 12 12 20 22 22 23 24 26 26
26 27 28 28 29 29 32 33 34 34 36 36 36 36 39 39 40 41 46 50
50 50 52 53 54 54 55 56 56 58 59 59 59 59 60 60 61 61 63 64
64 65 66 67 67 67 68 68 73 74 74 74 75 76 78 78 79 79 80 80
80 84 85 85 86 86 86 87 87 88 88 89 91 93 93 93 93 95 97 99
```

Observations	The merge sort algorithm contains two functions, which are merge() for merging the temporary arrays from left to right, and mergeSort() for the main sorting the first and second half recursively. The merge sort uses recursion to sort the two halves and calls the merge function to merge the two halves into one array. It has an overall time complexity of O(n log n) whether the array is already sorted or randomly ordered.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

```
template<typename T>
int partition(T arr[], int low, int high){
    // choose pivot
    T pivot = arr[high];
    // right index position of pivot
    int i = low - 1;
    // traverse low to high, move smaller elements to left
    for(int j = low; j < high; j++){
        if(arr[j] <= pivot){
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    // move pivot after smaller elements
    std::swap(arr[i + 1], arr[high]);
    // return position
    return i + 1;
}
```

```
template<typename T>
void quickSort(T arr[], int low, int high){
    if(low < high){
        // partition return index
        int pivot = partition(arr, low, high);
        // recursively call for small
        quickSort(arr, low, pivot - 1);
        // for greater or equal
        quickSort(arr, pivot + 1, high);
    }
}

#endif
```

```
    std::cout << "\nQuick Sort:\n";
    quickSort(quickArr, 0, maxSize - 1);
    displayArr(quickArr);
```

```
Quick Sort:
0 2 3 4 5 5 8 9 10 11 11 12 12 20 22 22 23 24 26 26
26 27 28 28 29 29 32 33 34 34 36 36 36 36 39 39 40 41 46 50
50 50 52 53 54 54 55 56 56 58 59 59 59 59 60 60 61 61 63 64
64 65 66 67 67 67 68 68 73 74 74 74 75 76 78 78 79 79 80 80
80 84 85 85 86 86 86 87 87 88 88 89 91 93 93 93 93 95 97 99
```

Observations	The quick sort algorithm has two functions, which are the partition and quick sorting function. The partition function is used to find the pivot, move the index position, traverse from low to high, and move the smaller elements to the left. The sorting function takes the returned partition index to recursively sort for the smaller and greater or equal to the pivot value. It has a best case scenario of O(n log n) when the pivot element divides the array evenly, while

	its worst case scenario is O(n^2) when the pivot is not an average element and is either the largest or smallest element.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms?
The left and right sublist can be sorted using other algorithms, such as insertion sort for sorting the sublists with smaller elements. Other algorithms can be used in place of using the standard recursive method of the quick sort algorithm. What makes this possible to use other algorithms is the use of partitioning for separating the array into two sublists where the values smaller than the pivot are moved to the left and conversely for the larger values.

Demonstrate an example.
For example, consider an unsorted array of 9 elements [54, 27, 92, 18, 76, 32, 43, 57, 21].

To divide the list, use 54 as the pivot for the partitioned list to divide the array into two partitions
[27, 18, 32, 43, 21] for values lower than the pivot and [92, 76, 57] for values higher than the pivot.

Then, insertion sort can be used to sort these two sublists, so the sorted subarrays should now be
[18, 21, 27, 32, 43] and [57, 76, 92].

After this, the divided subarrays are merged into the final sorted array by merging the left, pivot, and right arrays.
[18, 21, 27, 32, 43, 54, 57, 76, 92].

Table 9-1. Problem 1 Solution

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}.

What sorting algorithm will give you the fastest time performance?
The algorithm that has the fastest performance for this given array is Merge Sort. Considering that this array contains 15 unsorted elements, which could take a high number of iterations to sort using other sorting methods. With this algorithm, it has a consistent time complexity of O(n log n) across best, worst, and average case scenarios. Quick Sort is also another good choice for the fastest performance but its best performance depends on the proportions from the division around the pivot. If the pivot value leads to unbalanced partitioned arrays, its time complexity could result in O(N^2).

Why can merge sort and quick sort have O(N • log N) for their time complexity?
Since both merge and quick sort use the Divide and Conquer method by dividing or partitioning the array into parts, sorting them into the correct order, and combining them, they result in the same time complexity. In merge sort, the array is divided based on the value of log N times. For each iteration, the elements inside the subarrays are compared and then merged back.. It takes N times for comparing and sorting the values, and then merging them back to the full array. Hence, the given time complexity of O(N • log N). Similar to merge sort, quick sort

partitions the array into two proportional halves based on the comparison of the values to the selected pivot, which recursively divides the array into smaller subarrays and compares them. This process takes N times for each comparison in the subarrays with the pivot value, while the division takes log N times based on the proportions of the divided arrays.

Table 9-2. Problem 2 Solution

Conclusion

This activity has successfully demonstrated a new set of sorting algorithms that utilize “Divide and Conquer” methods and recursive sorting techniques to achieve the O(n log n) time complexity for most scenarios. This is especially important in larger data structures, such as the 100 element array of randomly generated values used in the procedure activities. These following algorithms used in the activity are shell, merge, and quick sort. The procedures use these three algorithms to solve the unsorted data in a given array. For consistency and reproducible results, a predetermined seed value is used throughout the tests of each sorting algorithm. Each algorithm has its own characteristics shown in the code. For example, both merge and quick sort use a recursive sorting technique but have different approaches in dividing, comparing, and sorting the array. One halves the arrays and compares each element individually for each subarray, while the other uses a pivot to divide the array and compare the values in the subarrays. The supplementary activities demonstrated that it is possible to sort the divided sublists with other sorting techniques alongside the main sorting algorithm. A notable example is using the quick sort for a quick division of the array by using its pivot, and then using insertion sort to manage the sorting of the smaller sublists to optimize the sorting process. Overall, this activity has introduced and demonstrated these efficient sorting algorithms that target an O(n log n) time complexity by implementing each algorithm into working functions and their respective helpers. It has also illustrated the differences between the quick and merge sort in terms of their behavior in sorting, and how the proportions of the array affect their performance with visualizations shown in the discussions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, clarification of definitions**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to clarify the more abstract concepts shown in the discussion of this activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.